

Códigos binarios

Práctica

2025

Temario

- 1- Códigos binarios
- 2- Códigos alfanuméricos
- 3- Códigos para detección de errores
- 4- Códigos para corrección de errores

1- Códigos binarios

Definición: Los códigos binarios son representaciones de información en forma de combinaciones de bits.

Aplicación: Son fundamentales en sistemas digitales para codificar datos.

1.2- Códigos BCD natural

Definición: El código BCD (del inglés Decimal Codificado en Binario), es un código donde cada dígito decimal se representa como un grupo de 4 bits.

Con ponderación **8 4 2 1**

Ejemplo: El número 59 se codifica como 0101 1001 en BCD.

Decimal	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

1.2- Códigos BCD Aiken

Definición: El código BCD Aiken es un código autocomplementario

Los autocomplementarios son diseñados para que el complemento a la base reducida sea evidente a partir de su representación binaria.

Con ponderación 2 4 2 1

Ejemplo: El número 59 se codifica como 1011 1111 en Aiken.

Decimal	Aiken
0	0000
1	0001
2	0010
3	0011
4	0100
5	1011
6	1100
7	1101
8	1110
9	1111

1.3- Códigos no ponderados Exceso-3

Definición: El código XS-3 es un código autocomplementario que representa cada dígito sumándole 3.

Así, el dígito 7 se codifica como $(7+3)$ 1010.

Ejemplo: El número 59 se codifica como 1000 1100 en XS-3.

Decimal	XS-3
0	0011
1	0100
2	0101
3	0110
4	0111
5	1000
6	1001
7	1010
8	1011
9	1100

1.3- Códigos continuos

Definición: Un código binario es continuo si sus combinaciones correspondientes a los números decimales consecutivos son adyacentes.

Se denominan combinaciones binarias adyacentes a aquellas que difieren solamente en un bit.

El código binario natural como el BCD natural **NO** son adyacentes

1.4- Códigos Gray (no ponderado)

Comúnmente llamado “reflejado”

Para crearlo tomo el 0 y el 1. Agrego un 1 a la izquierda, y reflejo los anteriores. Y así sucesivamente...

Cada número consecutivo difiere del anterior en un solo bit, lo que reduce errores en transmisiones.

Decimal	Gray
0	0000
1	0001
2	0011
3	0010
4	0110
5	0111
6	0101
7	0100
8	1100
9	1101
10	1111
11	1110
12	1010
13	1011
14	1001
15	1000

1.4- Conversión Código Binario a Gray

Metodo de conversion

El MSB del binario coincide con el MSB del Gray.

Luego de izq a der, sumar cada par de bits adyacentes en el código binario y descartar el acarreo. El resultado es el bit en código Gray.

Ejemplo Guia 1 19-a: Convertir el número binario 11011_2 a código Gray

1	1	0	1	1
↓				
1				

1.4- Conversión Código Binario a Gray

Metodo de conversion

El MSB del binario coincide con el MSB del Gray.

Luego de izq a der, sumar cada par de bits adyacentes en el código binario y descartar el acarreo. El resultado es el bit en código Gray.

Ejemplo Guia 1 19-a: Convertir el número binario 11011_2 a código Gray

 1 0 1 1
1

1.4- Conversión Código Binario a Gray

Metodo de conversion

El MSB del binario coincide con el MSB del Gray.

Luego de izq a der, sumar cada par de bits adyacentes en el código binario y descartar el acarreo. El resultado es el bit en código Gray.

Ejemplo Guia 1 19-a: Convertir el número binario 11011_2 a código Gray



$$1+1 = 10 \Rightarrow 0$$

1.4- Conversión Código Binario a Gray

Metodo de conversion

El MSB del binario coincide con el MSB del Gray.

Luego de izq a der, sumar cada par de bits adyacentes en el código binario y descartar el acarreo. El resultado es el bit en código Gray.

Ejemplo Guia 1 19-a: Convertir el número binario 11011_2 a código Gray



1.5- Código Johnson

La capacidad de codificación de un código Johnson de n posiciones es de $2n$ cantidades diferentes.

Ejemplo con 5 bits

Decimal	Johnson
0	00000
1	00001
2	00011
3	00111
4	01111
5	11111
6	11110
7	11100
8	11000
9	10000

2- Códigos alfanuméricos

Código ASCII (del inglés: código estándar americano para intercambio de información)

Ejemplo: Busque la tabla ASCII y verifique que la frase "Hola" se codifica como:

H: 01001000_b (48_h)

o: 01101111_b (6F_h)

l: 01101100_b (6C_h)

a: 01100001_b (61_h)

Código Unicode: Extiende a ASCII para representar caracteres de diferentes lenguas y símbolos. Ejemplo: El símbolo © (copyright) se representa como U+00A9.

3- Códigos para detección de errores

3.0 Definición

3.1 Función Impar

3.2 Paridad

3.3 Generador de paridad

3.4 Verificación

3.5 Ejercicio tipo

3.0- Códigos para detección de errores

Definición: Los códigos de detección de errores permiten identificar errores que pueden ocurrir durante la transmisión o almacenamiento de información.

Aplicación: Para garantizar la integridad de los datos.

3.1 Función Impar

Es una función que “cuenta” la cantidad de ‘1’s de una palabra binaria, devolviendo un bit activado si la cantidad de ‘1’s es impar.

Por ejemplo: “01011101” -> ‘1’

3.2- Paridad

El bit de paridad es un bit extra que se agrega a la derecha del mensaje para hacer que el número total de bits 1 en el mensaje sea par (paridad par) o impar (paridad impar)

3.2- Tipos de paridad

Existen dos tipos de paridad:

Paridad Par: genera un bit adicional tal que, el total de bits activos sea par.

por el contrario

Paridad Impar: genera un bit adicional tal que, el total de bits activos sea impar.

3.3- Generador de Paridad Par

Para generar la Paridad Par usamos la función Impar.

La cual agrega un bit en '1' cuando la cantidad de bits es impar, llevando el total de bits a par.

En nuestro ejemplo lo hacemos con tres bits.

Tabla de verdad del Generador de Paridad Par ->

d2	d1	d0	pp
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

3.4- Verificación

Siguiendo con nuestro ejemplo:

Ya que la información se transmitió con Paridad Par, los cuatro bits recibidos deberán tener un número par de unos.

En consecuencia, si los cuatro bits recibidos **NO** tienen un número par de unos, quiere decir que hubo un error de transmisión. (un bit habrá cambiado de valor durante la transmisión).

La salida del verificador de Paridad Par, denotada por E, será igual a 1.

3.4.1 Verificador de Paridad Par

Para verificar la Paridad Par usamos nuevamente la función impar.

Tomamos el ejemplo anterior.

Tabla de verdad del Verificador de Paridad Par ->

d2	d1	d0	pp	E
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1

1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

3.5- Ejercicio tipo

GTP_01

20- Determinar si el siguiente código con paridad par es erróneo.

e- 1001001110_2

Solucion:

Paso 1: Tomamos el bit de paridad y lo separamos del mensaje.

$100100111-0$

Paso 2: Tomamos el mensaje y le aplicamos la función impar.

$100100111 \rightarrow 1$

Paso 3: Si el resultado de la función coincide con el bit de paridad el dato es válido. Caso contrario el mensaje es erróneo.

$0 \neq 1$ código erróneo

4- Códigos para corrección de errores

4.1- Repaso teórico Código Hamming

4.2- Codificación Hamming

4.3- Decodificación Hamming

4.4- Ejemplo

4.1- Repaso teórico

Dado un mensaje de ' n ' bits. Codificar en hamming significa agregar ' p ' bits de verificación. Obteniendo un código de $n+p$ bits.

Dado que con 2^p combinaciones, una sola indica que el código es correcto; nos quedan 2^p-1 combinaciones para detectar el error.

Luego, se debe cumplir: $n+p \leq 2^p-1$

4.2- Codificación Hamming

El código se forma con una sucesión de bits de izquierda a derecha numerados de 1 a $n+p$. ($n=4$)

Los bits p se ubican en los pesos con potencia de 2, es decir posiciones 1,2,4,...

Los bits n se ubican en las restantes posiciones, de menos significativo a más significativo.

Posición bits	b1	b2	b3	b4	b5	b6	b7
Mensaje			$d1$		$d2$	$d3$	$d4$
	p1		$d1$		$d2$		$d4$
		p2	$d1$			$d3$	$d4$
				p3	$d2$	$d3$	$d4$
Cod Hamming	p1	p2	$d1$	p3	$d2$	$d3$	$d4$

4.2- Codificación Hamming

Cada bit p determina la **paridad par** de los siguientes bits:

$p1 \leq \text{bits } (3, 5, 7)$

$p2 \leq \text{bits } (3, 6, 7)$

$p3 \leq \text{bits } (5, 6, 7)$

Quedando así conformado el código.

4.3- Decodificación Hamming

Al recibir el código, verificamos que no contenga error.

Obtenemos los bits de comprobación (**paridad par**) con los mismos bits que recibimos más su bit de paridad que generamos en el transmisor.

$c1 \leq \text{bits}(1, 3, 5, 7)$

$c2 \leq \text{bits}(2, 3, 6, 7)$

$c3 \leq \text{bits}(4, 5, 6, 7)$

Si $c1=c2=c3=0$, la paridad par es correcta y no hay error.

4.3- Decodificación Hamming

Sino, c_1 , c_2 , y c_3 determinan, en binario, el bit que tiene el error.

Recordemos:

Si $c_1=c_2=c_3=0$, la paridad par es correcta y no hay error.

c3	c2	c1	Pos
0	0	0	0
0	0	1	1
0	1	0	2
0	1	1	3
1	0	0	4
1	0	1	5
1	1	0	6
1	1	1	7

4- Ejemplo

25 - Verificar si la siguiente secuencia que fue transmitida utilizando el código de Hamming con paridad par presenta algún error. En caso afirmativo, corregirla.

c. 0111001_2

Solución:

Cálculo la paridad par de los bits de corrección.

$c1 = b1, b3, b5, b7 = 0, 1, 0, 1 = 0$

$c2 = b2, b3, b6, b7 = 1, 1, 0, 1 = 1$

$c3 = b4, b5, b6, b7 = 1, 0, 0, 1 = 0$

Como el resultado es $= 010 = 2$. Hay un error en el bit2, corregido da 0011001_2 .
Como el error está en un bit de paridad en este caso no afecta al mensaje.

Preguntas ?